Hackathon

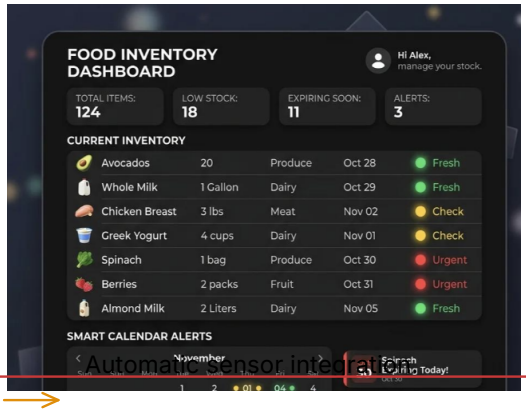
Sun 11 Oct 2026 11:24 AM

MVP

1. Identify a problem
Build a solution
Test and improve
Present the solution

24Hrs
One Day
Seven Day
Three Days





Enter today's food inventory.

Estimate surplus food.

Generate a recommendation.

Notify the cafeteria manager.

Automatic sensor integration.

Demand forecasting.

Multiple cafeteria support.

Advanced analytics and reporting.

Project A

Smart Food Waste Manager

Problem: Cafeterias prepare more food than they need.

Solution: Analyse past consumption, estimate demand, flag likely surplus, and recommend preparation quantities.

Project B

Student Learning Support Assistant

Problem: Students struggle to identify which topics to revise.

Solution: Analyse quiz results and recommend learning resources based on incorrect answers.

Students struggle to identify which topics they need to revise.

Analyse quiz results and recommend learning resources based on incorrect answers.

Project C

Smart Energy Saver

Problem: Offices and apartment buildings waste electricity.

Solution: Analyse electricity readings, detect unusual usage, and recommend energy-saving actions.

Project D

Waste Collection Prioritiser

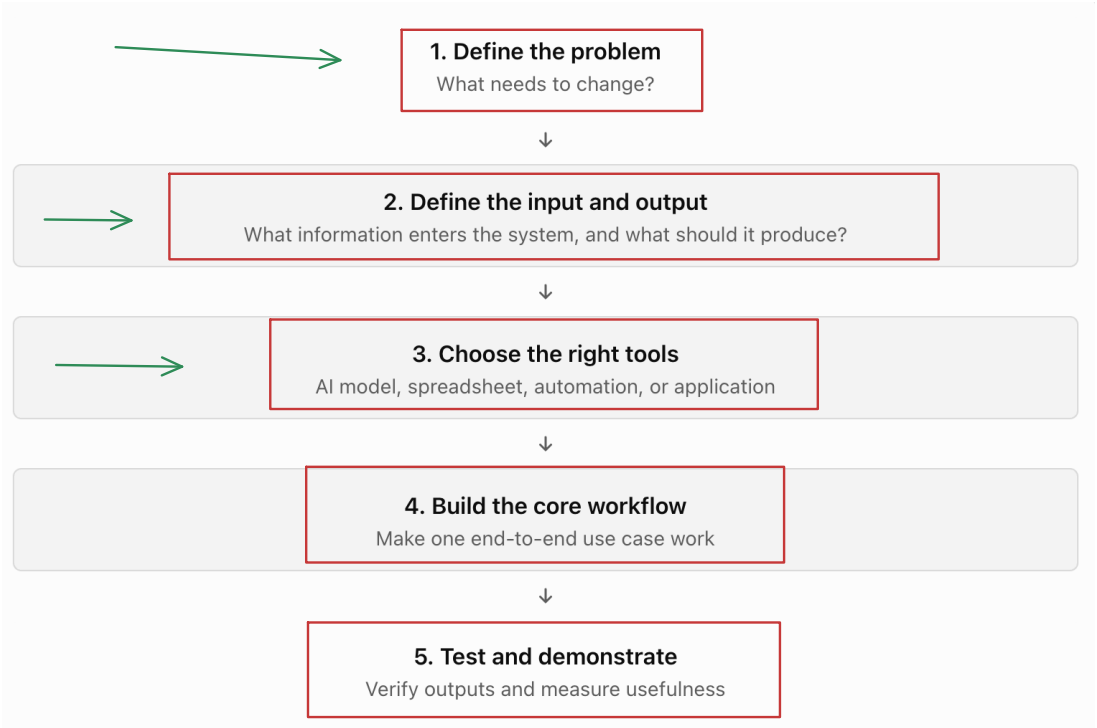
Problem: Collection teams may not know which locations need urgent attention.

Solution: Prioritise reports by fill level, time since collection, and location.

Question	What you're checking
Who has the problem?	The target user
What is the current difficulty?	The actual pain point
How is it solved today?	The existing process
Where does AI help?	The role of intelligence
What measurable benefit will result?	The expected impact

student-support chatbot

Our project helps [target user] solve [specific problem] by using [AI or automation capability], resulting in [measurable benefit].



ChatGPT / Claude / Gemini	Brainstorming, generating code, designing prompts, analysing data
Google Sheets / Excel	Storing and analysing structured information
n8n	Connecting services and automating workflows
Streamlit	Creating a simple Python-based interface
Telegram / Email	Sending notifications
Weather or other public APIs	Retrieving external data when required

Act as a product manager and AI solution architect.

I want to build an MVP for a Smart Food Waste Management System for a college cafeteria.

The problem is that cafeteria managers struggle to estimate how much food to prepare each day, resulting in avoidable surplus.

Design an MVP with the following sections:

1. Target users and their pain points.
2. Three essential features only.
3. Required input data and expected outputs.
4. Where AI is genuinely useful and where ordinary calculations are sufficient.
5. Recommended technology stack for a beginner-friendly prototype.
6. Step-by-step implementation plan.
7. Five test cases, including unusual or missing input data.
8. Three measurable success metrics.

Do not assume that historical data is available. Clearly distinguish assumptions from facts, and do not invent measured results.

Keep the first version small enough to demonstrate in a short

hackathon.

Role: Establishes the perspective of a product manager and solution architect.
Context: Defines the cafeteria and its specific problem.
Constraints: Limits the MVP to three essential features.
Output format: Requests an implementation plan, test cases, and metrics.
Validation: Requires assumptions to be disclosed rather than presented as facts.

Seven Day Plan - Hackathon

Build for Good Hackathon 2026

Hackathon	Be10X — Build for Good
Dates	11–19 October 2026
Format	Individual, self-paced, mostly no-code
Deadline	19 October 2026, 11:59 PM
Registration	No separate registration; submitting the form enters you
Minimum submission	One completed project

three tracks

Track 1 — AI Automation

Track 2 — AI Agent

Track 3 — Product Build

AI leverage	**30/100**
Does AI genuinely perform work that would otherwise require human interpretation or effort?	
Problem framing	**25/100**
Is the problem specific, real and clearly connected to a person or community?	
Practical usefulness	**20/100**
Does the solution save time, reduce cost or improve outcomes?	
Execution quality	**15/100**
Does the solution work from input to output?	
Clarity of explanation	**10/100**
Can you demonstrate and explain it clearly?	

Bonus: Up to 5 points for building with or testing with a real NGO or affected person.

Penalty: –10 points for exposing real personal data in the demo.

Penalty: –10 points if the project makes a final decision about someone's health, money, or legal rights without human approval.

A. Disaster and civic services

- Flood and heat alerts
- Relief supply matching
- Pothole complaint filing
- Water contamination reporting
- Compensation claim guidance

B. Welfare and accessibility

- Widow welfare scheme finder
- Veterans' pension navigator
- Form reader for blind users
- Disability certificate guidance
- Sign-language caption assistant

C. Elders, orphans and literacy

- Medicine reminders with escalation
- Daily check-ins for elders
- Senior citizen benefit finder
- Voice-first literacy tutor
- Read-aloud form explainer

D. Animals, environment and livelihoods

- Street-animal rescue dispatcher
- Lost-pet matching
- Waste-segregation assistant
- Tree-survival tracker
- Voice résumé generator
- NGO report and receipt generator

Build for Good Hackathon: Pothole Complaint Filing AI

Residents often encounter potholes but struggle to identify the correct municipal department, write a formal complaint, provide the right information, and follow up when no action is taken.

PotholeAI

AI-powered civic complaint filing and follow-up assistant.

Target users

Residents, housing societies and community volunteers who report damaged roads.

AI contribution

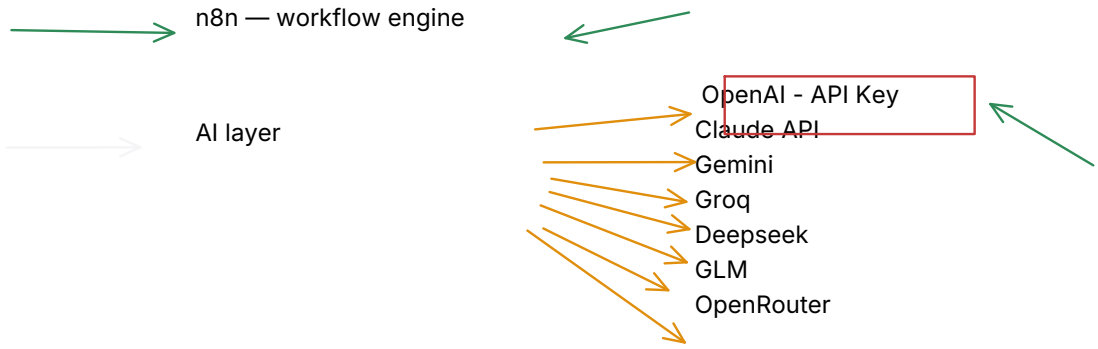
Interpret a photo and natural-language description, classify the road defect, assess apparent severity, and draft a formal complaint.

Automation contribution

Log complaints, assign a tracking ID, notify the user and schedule follow-up reminders.



Technology stack for your Mac/Windows - Linux



You are an AI civic complaint assistant helping residents report road damage.

Analyse the supplied pothole photograph and the resident's description.

Return a structured JSON object containing:
- defect_type: pothole, cracked_road, exposed_manhole, or other
- severity: low, medium, high, or uncertain
- visual_evidence: observable characteristics supporting the classification
- safety_concern: a concise explanation of any visible risk
- location: the location supplied by the resident
- missing_information: details required before preparing the complaint
- suggestred_department: a possible responsible authority, only if supported by the supplied location and verified routing information
- complaint_title: a concise title
- complaint_draft: a professional, factual complaint
- confidence: high, medium, or low

Rules:
1. Describe only what can reasonably be observed or supported.
2. Never invent an address, GPS coordinate, municipal department, or incident date.
3. Do not infer precise dimensions or accident history from a photograph alone.
4. If the image is unclear, ask for another photo or human review.
5. Do not submit the complaint automatically.
6. Return valid JSON without additional commentary.

Act as a civic correspondence assistant.

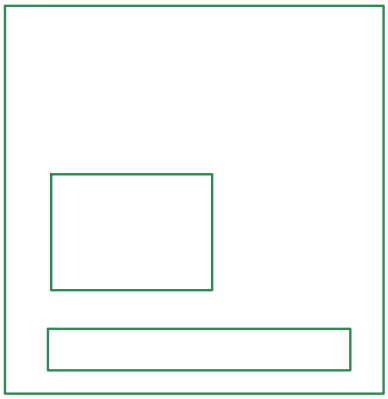
Using the verified information below, draft a formal road-damage complaint.

Inputs:
- Location: {{location}}
- Defect: {{defect_type}}
- Observed evidence: {{visual_evidence}}
- Safety concern: {{safety_concern}}
- Resident description: {{description}}
- Suggested authority: {{suggested_department}}

Produce:
1. Subject line.
2. Formal complaint body.
3. Requested action.
4. Evidence and attachment list.
5. Missing details requiring confirmation.

Requirements:
- Be factual, concise and respectful.
- Do not invent incident dates, accidents, injuries or previous complaints.
- Do not claim the authority has jurisdiction unless verified.
- If key details are missing, ask the resident to provide them.
- Mark the output as a draft requiring resident approval.

Telegram — user interfacea



Google Sheets — complaint database

Google Sheets database design

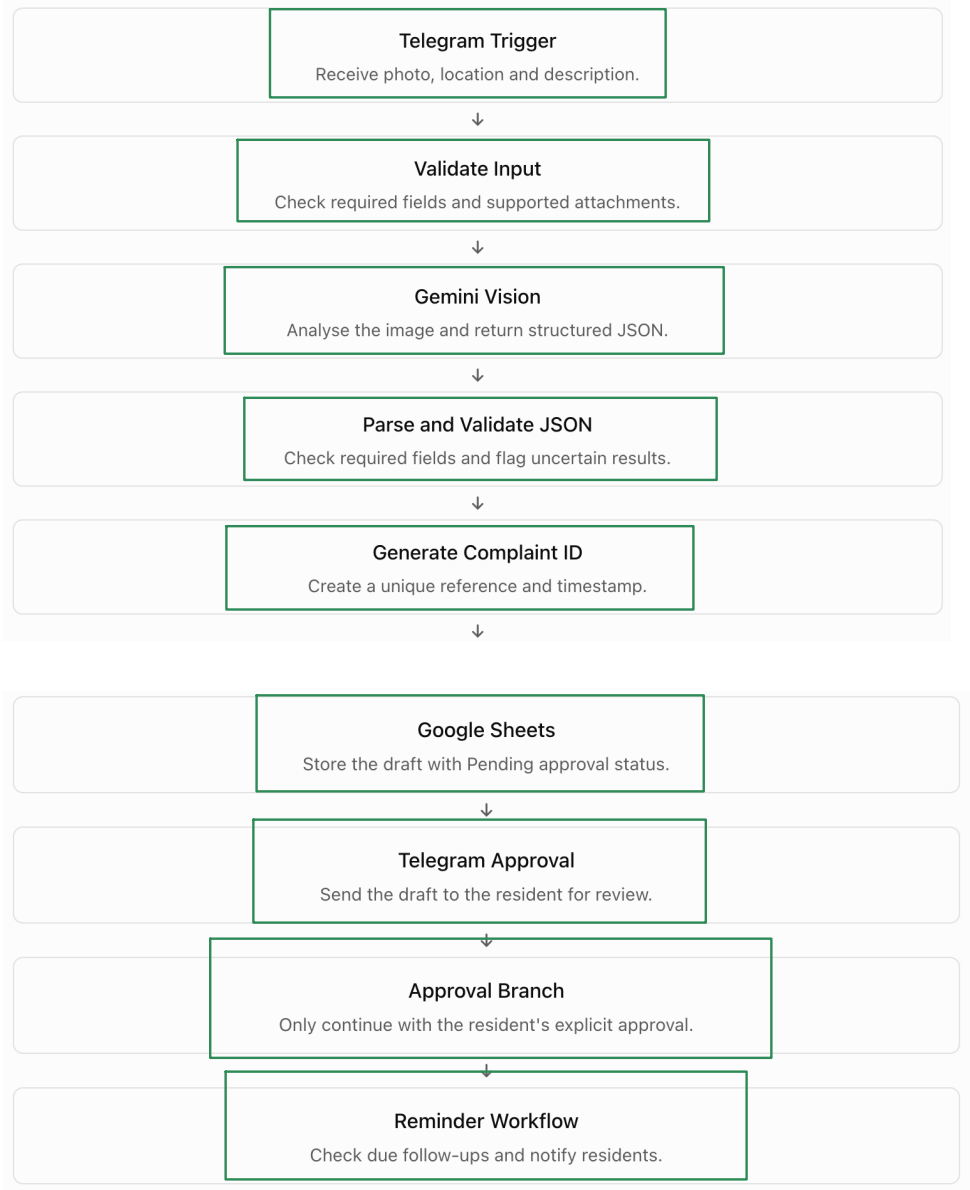
PotholeAI_Tracker

Complaintsa

Column	Purpose
Complaint_ID	Unique tracking reference
Created_At	Submission timestamp
Location	Resident-provided location
Description	Resident's description
Photo_Reference	Link or file reference to evidence
Defect_Type	AI-classified defect
Severity	AI assessment
AI_Confidence	Confidence category

Complaint_Draft	Generated complaint
Approval_Status	Pending, approved or rejected
Submission_Status	Draft, submitted or not submitted
Department	Verified authority, if known
Follow_Up_Date	Planned reminder date
Follow_Up_Status	Pending or completed

Flow With n8n





1. Open BotFather

Open Telegram and search for `@BotFather`, or use the official link:

Open BotFather ↗



2. Start the conversation

Send the command:

`/newbot`

3. Choose your bot name

Example: `AIsquad Automation Bot`

4. Choose a unique username

Example: `AIsquadAutomationBot`

The username must be available and end in `bot`.

5. Copy the bot token

BotFather will return an API token. Store it securely as a secret or environment variable. Anyone with this token can control your bot, so never share it publicly.

11 Oct	Pick one problem and define the user
12 Oct	Finalise workflow and select tools
13–14 Oct	Build the first working version
15 Oct	Test with dummy data and fix errors
16 Oct	Improve usability and measure impact
17 Oct	Prepare the explainer and rehearse
18 Oct	Record demo, test links and complete the form
19 Oct	Final checks and submit before 11:59 PM

First Place	₹30,000
Second Place	₹20,000
Third Place	₹10,000
Top 50 Scorers	3 months of free Gemini Enterprise access